Data Structure & Algorithm Lab

ECE2132



Department of Electronics & Communication Engineering

By:

Dr. D.V. Prashant
Dr. Kamal Kishore Upadhyay

Manipal University Jaipur,

Dehmi Kalan, Off Jaipur-Ajmer Expressway, Jaipur, (Raj.)

Rajasthan 303007

MANIPAL UNIVERSITY
JAIPUR
(University under Section 2(f) of the UGC Act)

(AY 2026-27)

Experiment No. 10

Write a Program to Implement Queue Using Arrays with Enqueue and Dequeue Operations

AIM: To study and implement a Queue using an array in C++ and perform Enqueue and Dequeue operations.

SOFTWARE USED: Visual Studio Code / Code: Blocks / Dev-C++, GCC/G++ Compiler. Operating System: Windows/Linux

THEORY: A Queue is a linear data structure in which insertion and deletion of elements are performed from different ends.

A Queue follows the FIFO (First In, First Out) principle. This means that the element inserted first is removed first.

A queue can be implemented using an array. Two variables, called front and rear, are used to keep track of the first and last elements of the queue.

Initially, front and rear are set to -1, which indicates that the queue is empty.

Queue

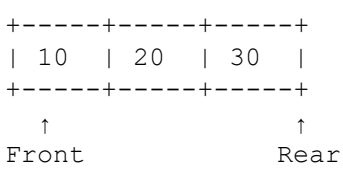
The queue performs two main operations:

Enqueue: The Enqueue operation is used to insert an element at the rear of the queue.

Dequeue: The Dequeue operation is used to remove an element from the front of the queue.

FIFO Principle

Queue follows the First In, First Out principle.



The element 10 will be removed first because it was inserted first.

Queue Overflow

Queue Overflow occurs when an attempt is made to insert an element into a queue that is already full.

Queue Underflow

Queue Underflow occurs when an attempt is made to remove an element from an empty queue.

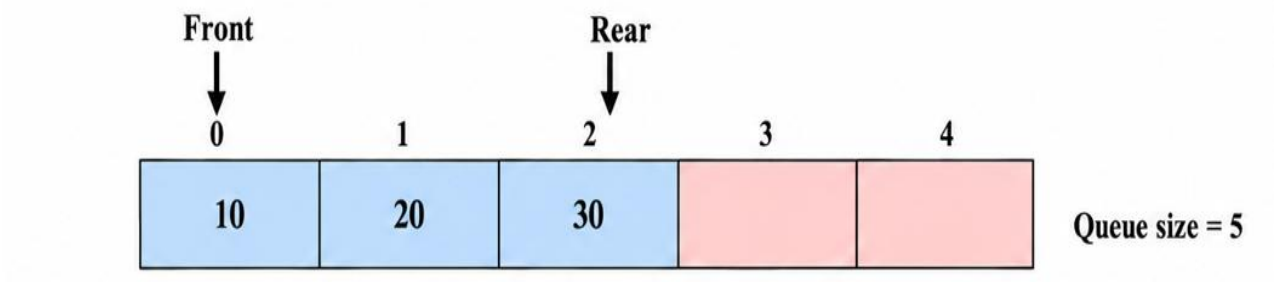


Figure.1: Queue representation using Array

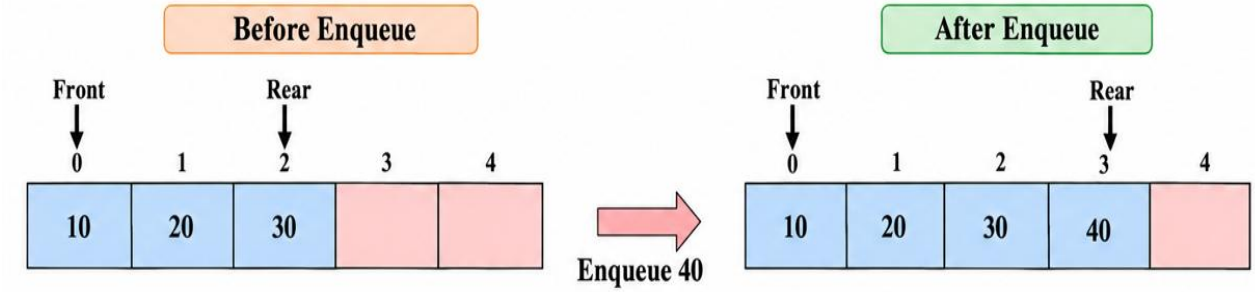


Figure.2: Enqueue Operation on Stack

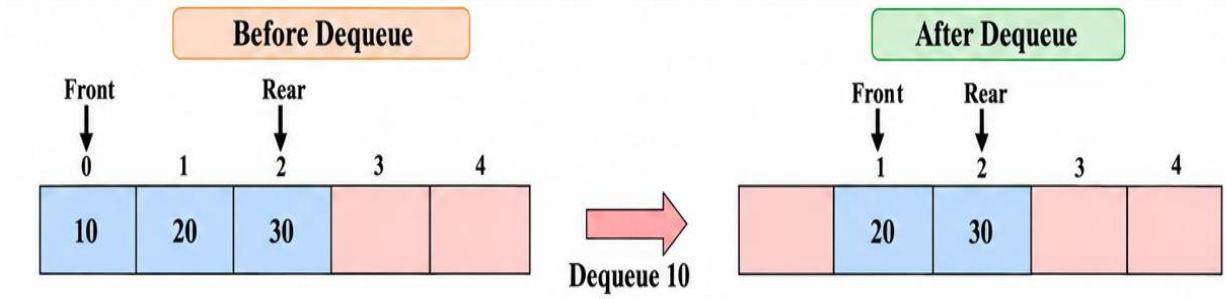


Figure.3: Dequeue Operation on Stack

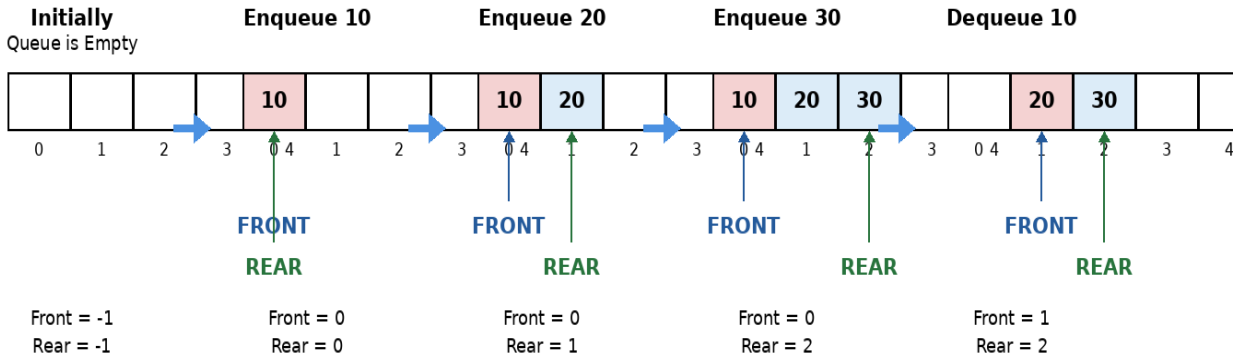


Figure.4: Working of Queue

Comparison of Queue Operations			
Feature	**Enqueue Operation**	**Dequeue Operation**	**Display Operation**
Purpose	Insert an element at rear	Remove the element from front	Display all queue elements
Condition to Check	rear == MAX - 1 (Queue Overflow)	front == -1 (Queue Underflow)	front == -1 (Queue Empty)
Time Complexity	O(1)	O(1)	O(n)
Space Complexity	O(1)	O(1)	O(1)
Effect on Pointer	rear = rear + 1	front = front + 1	No change
Can Operation Fail?	Yes (Overflow)	Yes (Underflow)	No

Figure. 5: Comparison of Queue operations

ALOGRITHM:

1. Start.
2. Declare an integer array queue and initialize front = -1 and rear = -1.
3. Display the menu containing Enqueue, Dequeue, Display, and Exit options.
4. Read the user's choice.
5. If the choice is Enqueue, check whether the queue is full.
6. If the queue is full, display Queue Overflow.
7. Otherwise, if the queue is empty, set front = 0.
8. Increment rear and insert the element into queue[rear].
9. If the choice is Dequeue, check whether the queue is empty.
10. If the queue is empty, display Queue Underflow.
11. Otherwise, display the element at queue[front].
12. If front == rear, reset front and rear to -1.
13. Otherwise, increment front.
14. If the choice is Display, display all elements from front to rear.
15. Repeat the operations until the user selects Exit.
16. Stop.

PROGRAM (C++):

```
#include <iostream>
using namespace std;

#define MAX 5
int queue[MAX];
int front = -1;
int rear = -1;

void enqueue(int value)
{
    if(rear == MAX - 1)
    {
        cout << "Queue Overflow!" << endl;
    }
    else
    {
        if(front == -1)
            front = 0;
        rear++;
        queue[rear] = value;
        cout << value << " inserted into queue." << endl;
    }
}

void dequeue()
{
    if(front == -1 || front > rear)
    {
        cout << "Queue Underflow!" << endl;
    }
    else
    {
        cout << queue[front] << " deleted from queue." << endl;

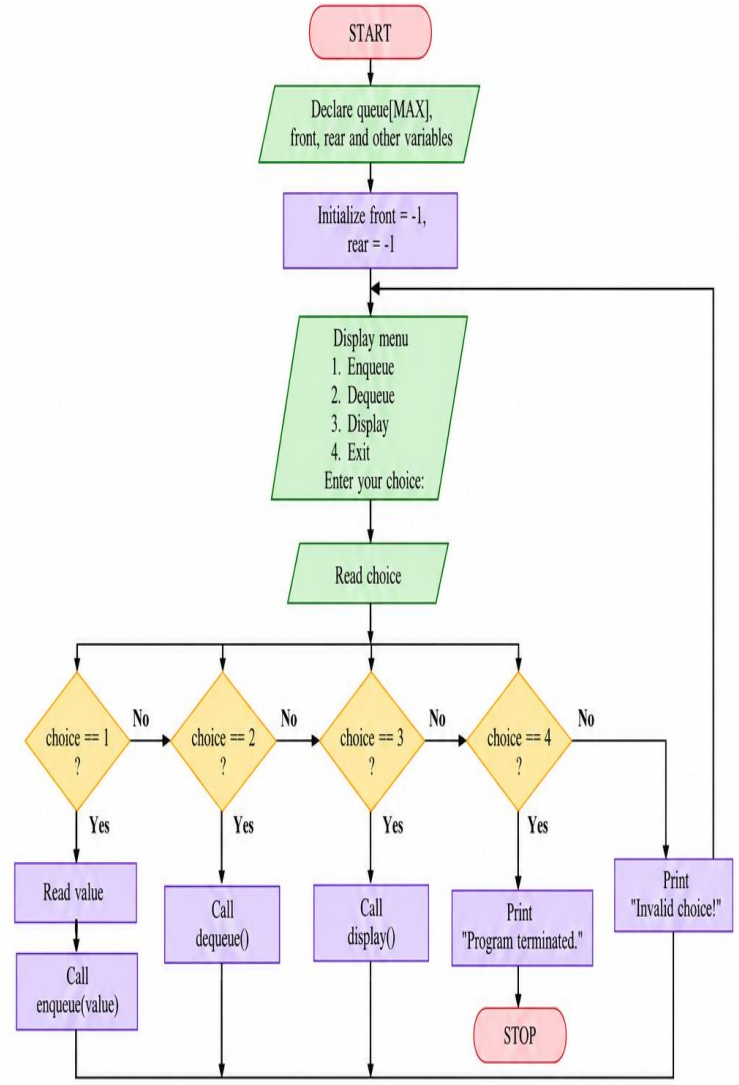
        if(front == rear)
        {
            front = -1;
            rear = -1;
        }
        else
        {
            front++;
        }
    }
}

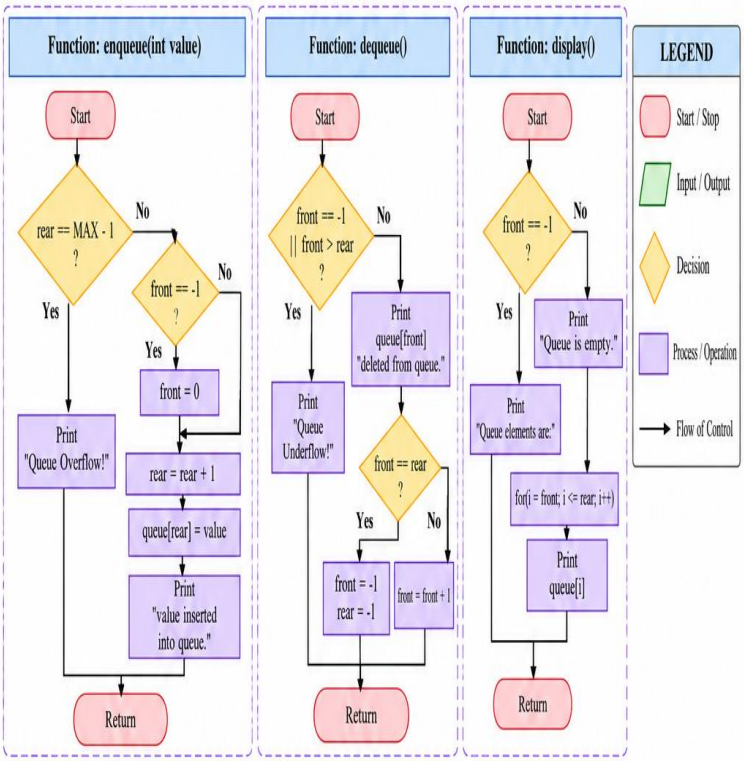
void display()
{
    if(front == -1)
    {
        cout << "Queue is empty." << endl;
    }
    else
    {
        cout << "Queue elements are:" << endl;
        for(int i = front; i <= rear; i++)
        {
            cout << queue[i] << " ";
        }
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Enqueue" << endl;
        cout << "2. Dequeue" << endl;
        cout << "3. Display" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if(choice == 1)
        {
            cout << "Enter value to enqueue: ";
            cin >> value;
            enqueue(value);
        }
        else if(choice == 2)
        {
            dequeue();
        }
        else if(choice == 3)
        {    display();
        }
        else if(choice == 4)
        {    cout << "Program terminated." << endl;
        }
        else
        {    cout << "Invalid choice!" << endl;
    } while(choice != 4);
}    return 0;
```





EXPLANATION OF THE PROGRAM:

Statement / Function	Explanation
MAX	Defines the maximum size of the queue.
queue[]	Stores the queue elements.
front	Stores the position of the first element.
rear	Stores the position of the last element.
enqueue()	Inserts an element at the rear.
dequeue()	Removes an element from the front.
display()	Displays all queue elements.
choice	Stores the operation selected by the user.
value	Stores the element to be inserted.
front == -1	Indicates that the queue is empty.
rear == MAX - 1	Checks whether the queue is full.

SAMPLE INPUT 1:

Enter your choice:

1

Enter value to enqueue:

10

Enter your choice:

1

Enter value to enqueue:

20

Enter your choice:

1

Enter value to enqueue:

30

Enter your choice:

3

CALCULATION:

Initial Queue:

Queue is empty

Front = -1
Rear = -1

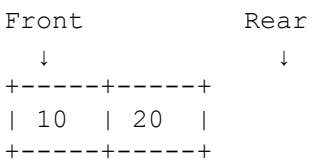
Enqueue 10



front = 0

rear = 0

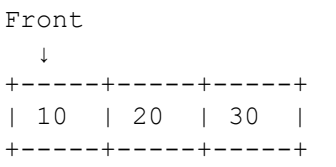
Enqueue 20



front = 0

rear = 1

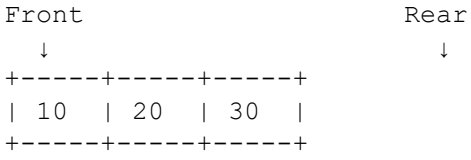
Enqueue 30



front = 0

rear = 2

Therefore, the final queue is:



SAMPLE OUTPUT:

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 10
10 inserted into queue.

Enter your choice: 1
Enter value to enqueue: 20
20 inserted into queue.

Enter your choice: 1

Enter value to enqueue: 30
30 inserted into queue.

Enter your choice: 3
Queue elements are:
10 20 30

SAMPLE INPUT 2:

After inserting:

10 20 30

Enter your choice:

2

CALCULATION:

The front element is 10.

Before Dequeue:

```
Front                    Rear
  ↓                        ↓
+-----+-----+-----+
| 10  | 20  | 30  |
+-----+-----+-----+
```

After Dequeue:

```
       Front               Rear
         ↓                   ↓
+-----+-----+
| 20  | 30  |
+-----+-----+
```

The element 10 is removed from the queue.

SAMPLE OUTPUT:

Enter your choice: 2
10 deleted from queue.

PERFORMANCE ANALYSIS:

The performance of queue operations depends on the operation being performed.

Enqueue Operation

The Enqueue operation inserts an element directly at the rear of the queue.

Time Complexity = O(1)

Dequeue Operation

The Dequeue operation removes the element directly from the front of the queue.

Time Complexity = O(1)

Display Operation

The Display operation visits all the elements of the queue.

Time Complexity = O(n)

TIME COMPLEXITY:

Enqueue: O(1)

Dequeue: O(1)

Display: O(n)

For a queue containing n elements, Enqueue requires one insertion operation, Dequeue requires one deletion operation, and Display requires visiting all n elements.

Therefore, Enqueue and Dequeue operations have constant time complexity.

SPACE COMPLEXITY:

The queue is implemented using an array of size MAX.

The array stores the queue elements.

Space Complexity = O(n)

CHARACTERISTICS OF QUEUE:

1. It is a linear data structure.
2. It follows the FIFO (First In, First Out) principle.
3. Insertion is performed from the REAR.
4. Deletion is performed from the FRONT.
5. Enqueue is used to insert an element.
6. Dequeue is used to remove an element.
7. Overflow occurs when the queue is full.
8. Underflow occurs when the queue is empty.
9. Enqueue and Dequeue operations have O(1) time complexity.
10. Queue can be implemented using an array.

IMPLEMENTATION OF CODE

```cpp
#include <iostream>
using namespace std;

#define MAX 5

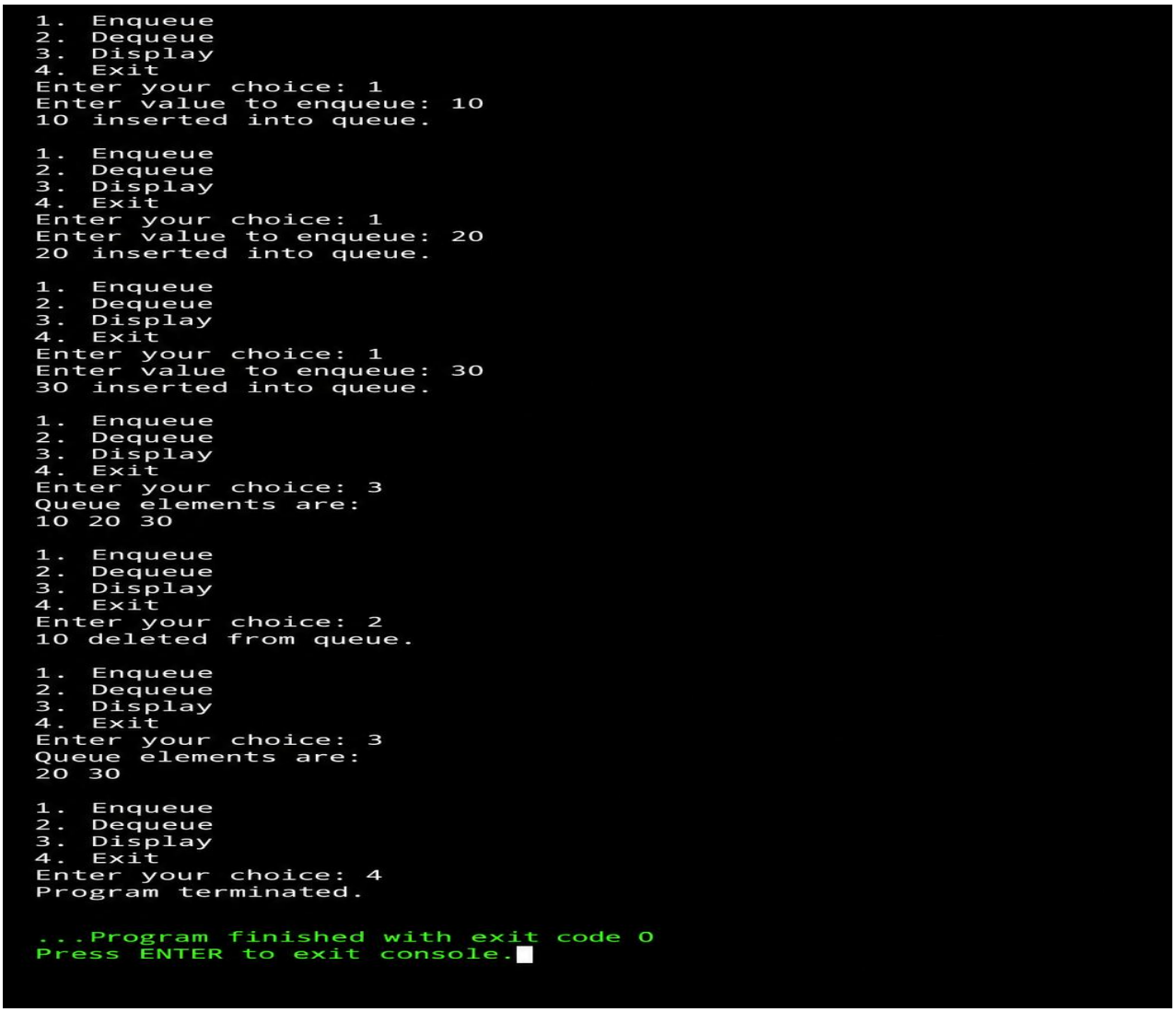
int queue[MAX];
int front = -1;
int rear = -1;

void enqueue(int value)
{
    if (rear == MAX - 1)
    {
        cout << "Queue Overflow!" << endl;
    }
    else
    {
        if (front == -1)
            front = 0;
        rear++;
        queue[rear] = value;
        cout << value << " inserted into queue." << endl;
    }
}

void dequeue()
{
    if (front == -1 || front > rear)
    {
        cout << "Queue Underflow!" << endl;
    }
    else
    {   cout << queue[front] << " deleted from queue." << endl;
        if (front == rear)
        {   front = -1;
            rear = -1;
        }
        else
        {   front++;
    }
}

void display()
{
    if (front == -1)
    {
        cout << "Queue is empty." << endl;
    }
    else
    {   cout << "Queue elements are:" << endl;
        for (int i = front; i <= rear; i++)
        {   cout << queue[i] << " ";
        }
        cout << endl;
    }
}

int main()
{
    int choice, value;
    do
    {   cout << "\n1. Enqueue" << endl;
        cout << "2. Dequeue" << endl;
        cout << "3. Display" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;
        if (choice == 1)
        {
            cout << "Enter value to enqueue: ";
            cin >> value;
        }   enqueue(value);
        else if (choice == 2)
        {   dequeue();
        else if (choice == 3)
        {   cout << "Program terminated." << endl;
        else
    } while (choice != 4);
}   return 0;
```



APPLICATIONS: Queue can be used in CPU scheduling, printer scheduling, task scheduling, buffering, keyboard input handling, network data transmission, and Breadth First Search (BFS).

RESULT:

The program was successfully executed. It demonstrated the implementation of a Queue using an array in C++ and performed Enqueue, Dequeue, and Display operations successfully. The concepts of FIFO, Queue Overflow, and Queue Underflow were also studied.

ASSIGNMENT:

- WAP to implement Queue using arrays with Enqueue and Dequeue operations.
- WAP to implement Queue using arrays with Peek operation.
- WAP to check whether the Queue is Empty or Full.
- WAP to implement Queue using arrays and display the front and rear elements.
- WAP to implement a Circular Queue using arrays.
- WAP to implement Queue using a linked list.
- WAP to compare Linear Queue and Circular Queue.
- WAP to reverse the elements of a Queue.
- WAP to implement a Queue for CPU scheduling.